QUEUE

ALGORITHM enqueue(student)

```
   IF rear = MAX_SIZE - 1 THEN
      PRINT "Queue is full"
      RETURN
   END IF
   IF front = -1 THEN              // queue was empty
      front ← 0
   END IF
   rear ← rear + 1
   queue[rear] ← student
END
```

ALGORITHM dequeue()

```
   IF front = -1 OR front > rear THEN
      PRINT "Queue is empty"
      RETURN null
   END IF
   student ← queue[front]
   front ← front + 1
   IF front > rear THEN            // queue became empty
      front ← -1
      rear  ← -1
   END IF
   RETURN student
END
```

STACK (Postfix Evaluation)

ALGORITHM push(value)

```
  IF top = SIZE - 1 THEN
     PRINT "Stack Overflow"
     RETURN
  END IF
  top ← top + 1
  stack[top] ← value
END
```

ALGORITHM pop()

```
  IF top = -1 THEN
     PRINT "Stack Underflow"
     RETURN error
  END IF
  value ← stack[top]
  top ← top - 1
  RETURN value
END
```

ALGORITHM evaluatePostfix(expression)

```
  CREATE empty stack
  FOR each token in expression DO
     IF token is a number THEN
        push(CONVERT token TO number)
     ELSE IF token is an operator (+, −, ×, ÷) THEN
```

```
        operand2 ← pop()
        operand1 ← pop()
        result ← APPLY operator TO (operand1, operand2)
        push(result)
      END IF
    END FOR
    RETURN pop()                    // final result left on the stack
END
```

SINGLY LINKED LIST

ALGORITHM insertNode(student, position)

```
    CREATE newNode
    newNode.data ← student
    newNode.next ← null

    IF position = 1 OR head = null THEN   // insert at beginning
      newNode.next ← head
      head ← newNode
      RETURN
    END IF

    current ← head
    count ← 1
    WHILE current ≠ null AND count < position - 1 DO
      current ← current.next
      count ← count + 1
```

```
    END WHILE

    IF current ≠ null THEN
        newNode.next ← current.next
        current.next ← newNode
    END IF
END
```

ALGORITHM deleteNode(studentNumber)

```
    IF head = null THEN
        RETURN false
    END IF

    IF head.data.studentNo = studentNumber THEN
        head ← head.next
        RETURN true
    END IF

    current ← head
    WHILE current.next ≠ null DO
        IF current.next.data.studentNo = studentNumber THEN
            current.next ← current.next.next
            RETURN true
        END IF
        current ← current.next
    END WHILE
```

```
    RETURN false
END
```

ALGORITHM searchNode(studentNumber)

```
    current ← head
    WHILE current ≠ null DO
        IF current.data.studentNo = studentNumber THEN
            RETURN current.data
        END IF
        current ← current.next
    END WHILE
    RETURN null
END
```

ALGORITHM traverseList()

```
    current ← head
    IF current = null THEN
        PRINT "List is empty"
        RETURN
    END IF
    WHILE current ≠ null DO
        PRINT current.data
        current ← current.next
    END WHILE
END
```

SORTING ALGORITHMS

SELECTION SORT

ALGORITHM selectionSort(A, n)

```
FOR i ← 0 TO n - 2 DO
    minIndex ← i
    FOR j ← i + 1 TO n - 1 DO
        IF A[j] < A[minIndex] THEN
            minIndex ← j
        END IF
    END FOR
    IF minIndex ≠ i THEN
        SWAP A[i] WITH A[minIndex]
    END IF
END FOR
END
```

INSERTION SORT

ALGORITHM insertionSort(A, n)

```
FOR i ← 1 TO n - 1 DO
    key ← A[i]
    j ← i - 1
    WHILE j ≥ 0 AND A[j] > key DO
        A[j + 1] ← A[j]          // shift
        j ← j - 1
    END WHILE
```

```
    A[j + 1] ← key
  END FOR
END
```

MERGE SORT

ALGORITHM mergeSort(A, left, right)

```
  IF left < right THEN              // base case
    mid ← (left + right) DIV 2
    mergeSort(A, left, mid)
    mergeSort(A, mid + 1, right)
    merge(A, left, mid, right)
  END IF
END
```

ALGORITHM merge(A, left, mid, right)

```
   CREATE temporary leftArray and rightArray
  // copy left and right halves into temporary arrays
  i ← 0, j ← 0, k ← left
  WHILE i < size of leftArray AND j < size of rightArray DO
    IF leftArray[i] ≤ rightArray[j] THEN
      A[k] ← leftArray[i]
      i ← i + 1
    ELSE
      A[k] ← rightArray[j]
      j ← j + 1
    END IF
```

```
        k ← k + 1
    END WHILE
    // copy any remaining elements from leftArray or rightArray
END
```

QUICK SORT

ALGORITHM quickSort(A, low, high)

```
    IF low < high THEN
        pivotIndex ← partition(A, low, high)
        quickSort(A, low, pivotIndex - 1)
        quickSort(A, pivotIndex + 1, high)
    END IF
END


ALGORITHM partition(A, low, high)          // pivot-selection rule: last element
    pivot ← A[high]
    i ← low - 1
    FOR j ← low TO high - 1 DO
        IF A[j] ≤ pivot THEN
            i ← i + 1
            SWAP A[i] WITH A[j]
        END IF
    END FOR
    SWAP A[i + 1] WITH A[high]
    RETURN i + 1
END
```